

Risk Data Governance Lead — ответ на тестовое задание

Plata · Group IRM · Parts 1–5 · подготовлено 12 августа 2026
Русская версия для внутреннего просмотра. Итоговая подача — на английском.

Как готовился этот ответ. Приложения A–H описывают данные, которых у меня нет. Вместо рассуждений «как бы я поступил» я собрал в Snowflake рабочую песочницу, воспроизводящую описанный ландшафт: 10 баз, 23 схемы, 59 таблиц и 15 вью, 8.1 млн строк — с теми же зёрнами, знаковыми соглашениями и теми же граблями. Поверх неё построено governance-слои: каталог, канонический метрик-слой, DQ-мониторинг, реестры изменений и контекст-слой для LLM-агента. Все цифры ниже — **замеры**, а не оценки. Сборка заняла 2.67 кредита Snowflake (около 8 USD).

14.62%
канонический COR
за апрель 2026

3.24 п.п.
разрыв между
тремя расчётами

26
автоматических
проверок, все проходят

3.4k
токенов — весь
контекст для агента

ОТДЕЛЬНОЕ ПРИЛОЖЕНИЕ

Ревизия исходных материалов: 20 находок в приложениях A–I

Прежде чем оценивать ландшафт, я сплошь вычитал саму документацию — она выдана как «day-one documentation», и именно с её противоречиями новый аналитик столкнётся раньше, чем с данными. Каждая находка приведена с цитатой, файлом и номером строки.

20
находок

6
ведут к неверному числу

5
ссылок на файлы, которых нет

3
видны только прогоном
запросов

Среди критичных: запрос в Appendix C, подписанный «Correct COR%», возвращает **отрицательный** процент; свежесть portfolio_cor описана одновременно как T-1 и T-2; statement_id объявлен TEXT в одном приложении и Integer в другом; календарь праздников содержит дату, перенесённую с 2024 года. Разбор и — главное — **чем каждый класс таких ошибок ловится системно** (dbt, diff, агент-валидатор):
answer/source-review/source-review-ru.pdf.

Part 1. Обзор ландшафта данных

1.1 Какие риски governance я вижу и как они ложатся на DMBoK

Риск	Дисциплина DMBoK	Свидетельство из приложений
Метрика без канона	Data Quality Management, Metadata	Три команды считают COR по трём формулам. Ни одна не помечена как основная.
Соглашения живут в головах	Data Architecture, Metadata	Знаки, зёрна, обязательные фильтры описаны прозой. Приложение С прямо предупреждает: неверный знак «молча даёт неверный результат».
Изменения без следа	Change Management	Дропнута NPV_GEN5_PV_DETAILS ; переименована колонка фильтра; пересчитан Gen5v1. Ни одного механизма уведомления.
Каталог как проза	Data Catalog	Приложения читает человек. Ни машина, ни агент их не разбирают.
Владение не определено	Data Governance	Нигде не сказано, кто решает, каким должно быть число, и кого будить при сбое.
Референсные данные	Reference Data Management	Один продукт зовётся CC , TDC и account_type='CC '. Реально, но по условиям задания не приоритет.

Самый дорогой класс — не тот, где запрос падает, а тот, где он **отрабатывает и врёт**. Неверная колонка фильтра, забытый `QUALIFY`, перепутанный знак — всё это даёт правдоподобное число, которое доходит до правления.

1.2 Как остановить рост расхождений

Три приёма, все проверены на песочнице.

Первый: правило встраивается в объект, а не пишется рядом с ним. Это не теория — я на этом споткнулся. Первая версия семантической модели для Cortex описывала обязательный фильтр `statement_num >= 2` в комментарии к метрике. Вопрос на естественном языке получил **14.42%** вместо канонических **14.62%**: комментарий ничего не обеспечивает. После того как фильтр зашит внутрь агрегата, расхождение стало нулевым.

Второй: конвенция становится проверкой. Знаковые соглашения приложения C лежат в таблице `REF_SIGN_CONVENTION`, где у каждого правила есть поле с готовым предикатом. Из него автоматически собирается контроль качества. Правило, которое можно запустить, сильнее правила, которое надо помнить.

Третий: сборка не принимается без прогона утверждений. 26 проверок покрывают знаки, зёрна, календарь, аддитивность, дедупликацию, отсутствие дропнутых объектов. Одна строка даёт вердикт:

```
SELECT * FROM RISK_GOV.DQ.BUILD_ASSERTIONS WHERE NOT passed;
```

Контур поймал **меня самого**. Проверка требовала, чтобы дропнутого объекта не существовало, а позже я создал под тем же именем «надгробие» — вью с внятным перенаправлением вместо ошибки «object does not exist». Два моих решения противоречили друг другу, и заметил это не я. Надгробие — лучший дизайн, поэтому я уточнил проверку, а не ослабил дизайн.

1.3 Кто чем владеет и что это значит операционно

Владение — не ярлык, а три **разделимые** обязанности. Смешение их в одно «ответственное подразделение» и есть причина, по которой владение обычно фиктивно.

Роль	За что отвечает	Как проверяется, что роль реальна
Data owner	Что число <i>означает</i> ; утверждает изменение определения метрики	Есть подпись под <code>METRIC_DEFINITIONS</code>
Data steward	Операционное качество; получает алерт при падении проверки	Назван поимённо и получает уведомление
Technical owner	Пайплайн и SLA загрузки	Инцидент заводится на него

Принцип распределения: **владелец — тот, кто несёт последствия неверного числа**, а не тот, кто написал пайплайн. Отсюда конкретное закрепление.

Домен данных	Владелец (смысл)	Стюард (качество)	Технический владелец
Определения метрик риска: COR, EL LT, винтажи	Group IRM	Датамарт-аналитик 2	—
Витрины <code>dm.*</code> и <code>tableau.*</code>	Group IRM	Датамарт-аналитик 1	Group IRM
Слой ODS и загрузка из источников	DWH Engineering	Дежурный DWH	DWH Engineering
Провизии MxGAAP, регуляторная отчётность	Finance	Ведущий по провизиям	Group IRM
Модели Gen2 / Gen3 / Gen5	Risk Modelling	Владелец модели	Group IRM
Справочники и календарь	Group IRM	Датамарт-аналитик 2	DWH Engineering
Контекст-слой для агентов	Group IRM	Лид (эта роль)	Group IRM

Две развилки, которые стоит проговорить вслух. **Провизии MxGAAP:** считает их риск, но отвечает за них Finance — потому что подпись под регуляторной отчётностью ставит финансовый блок. **Модели:** витрина принадлежит риску, а смысл предсказания — моделированию; без этого разделения пересчёт модели становится ничьим изменением, что и произошло с Gen5v1.

Владение реально только тогда, когда назван **порог и действие при его нарушении**. В реестре у каждого объекта есть поле «что делать при нарушении SLA» — от «разбудить стюарда» до «эскалировать в офис CFO, это вход регуляторной отчётности». Формулировка «ticket to DWH, no page — эта таблица известна тем, что отстаёт» тоже

валидна: она честно фиксирует согласованный уровень сервиса вместо тревоги, которую всё равно будут игнорировать.

Отдельно: SLA для статементных таблиц измеряется **в рабочих днях** по мексиканскому календарю. Календарные дни дают ложную тревогу каждый понедельник — а это главный способ, которым мониторинг теряет доверие аудитории.

Part 2. Governance метрик и качество данных

2.1 Ответ на вопрос «какой у нас COR за апрель»

14.62% годовых по кредитным картам — по каноническому определению. Не потому, что это самое красивое число, а потому, что зерно расчёта совпадает с событием, на котором метрика определена.

Параметр канона COR_PCT_CC v1.0	Значение
Зерно	счёт × статемент
Модель	Gen3, полный тотал (включая 0-бакет)
Обязательный фильтр	<code>product_risks = 'CC' AND statement_num >= 2</code>
Знаменатель	<code>SUM(avg_balance_prnp_net)</code>
Аннуализация	<code>× 12</code>

Обоснование канона: резерв двигается, когда пропущен платёж, а платежи привязаны к выписке. Дневной снимок отвечает на другой вопрос — как двигался сток во вторник — и подмешивает календарные эффекты в кредитную метрику. Первый статемент исключается потому, что он структурно несёт нулевой COR: включая его, мы разбавляем коэффициент величиной, которая зависит от объёма выдач, а не от риска.

2.2 Мост: откуда берутся другие два числа

Спор трёх команд заканчивается не выбором «правильного» числа, а разложением разницы. Каждый шаг ниже меняет **ровно одно** методическое решение и пересчитывается из данных, поэтому остаток моста равен нулю по построению.

Шаг (апрель 2026, продукт CC)	Вклад	Итог
Канон — статемент, Gen3, <code>stmt ≥ 2</code> , ×12	—	14.62%
снят фильтр <code>statement_num ≥ 2</code>	−0.20	14.42%
зерно: статемент → день	−2.14	12.28%
из числителя убран 0-DPD бакет	−0.53	11.75%
аннуализация ×12 → ×365	+0.16	11.92%
= Portfolio Monitoring		11.92%
поколение модели Gen3 → Gen2	−0.54	11.38%
= Finance		11.38%

2.3 Две находки, меняющие постановку задачи

Расхождение не имеет постоянного знака. В апреле канонический расчёт даёт самое высокое значение, в марте — самое низкое.

Месяц	Risk Analytics	Portfolio Monitoring	Finance	Разрыв
2026-02	13.59	12.95	13.69	0.74
2026-03	3.13	5.02	5.42	2.30
2026-04	14.62	11.92	11.38	3.24
2026-05	6.54	4.99	5.67	1.55
2026-06	12.92	10.25	8.56	4.37

Практическое следствие: договорённость вида «у Finance всегда на два пункта меньше, прибавляйте» **невозможна**. Нужен один канон, а не поправочный коэффициент. Это свойство закреплено отдельной проверкой, которая упадёт, если расхождение вдруг станет односторонним.

Самая известная грабля оказалась третьестепенной. Приложения C и D настойчиво предупреждают о фильтре `statement_num >= 2`. По замерам его вклад составляет от 0.05 до 0.24 п.п., тогда как выбор зерна даёт от -2.14 до +2.72 п.п. — на порядок больше. Предупреждение верное, но приоритет в документации расставлен неверно.

2.4 Как выравнивать команды

- 1. **Не удалять легаси-расчёты.** Удаление никого не мигрирует — оно переносит прежний расчёт в Excel, где его уже не видно. Старые определения помечены *deprecated* и остаются работоспособными.
- 2. **Мост как инструмент переговоров.** Команды перестают сравнивать два числа и начинают читать разложение разницы. Спор о том, кто прав, превращается в разговор о том, какое методическое решение принять.
- 3. **Канон живёт в вью и в семантической модели одновременно**, так что ни SQL-запрос, ни вопрос на естественном языке не могут сочинить свою формулу.

2.5 Детекция дрейфа против лестницы DQM

Степень	Что делается	Состояние в песочнице
Basic → Defined	Штатные метрики качества Snowflake на критичных таблицах: свежесть, пустые ключи, дубликаты, ссылочная целостность	12 привязок на 3 таблицах, суточное расписание
Defined → Controlled	Собственные проверки на <i>смысл</i> , а не на форму: знак в агрегате, аддитивность декомпозиции, отсутствие выходных в статементных датах	2 собственные функции качества; первая ловит нарушение календаря, вторая — инверсию знака
Controlled → Continuing	Светофор по семи измерениям, владелец и действие в той же строке, тренд по неделям	22 проверки: 21 зелёная, 1 жёлтая

Почему проверка знака сделана агрегатной, а не построчной. Отдельные положительные значения COR законны: это высвобождение резерва, когда счёт излечивается. Построчное правило срабатывало бы на каждом здоровом портфеле, и его бы отключили в первую неделю. Инверсия знака имеет смысл только в агрегате.

Единственная жёлтая проверка — реальный инцидент: `stm_cc` отстаёт на 9 рабочих дней при SLA 5. Показательно, что изначально я заложил это отставание только в журнал ETL, а сами данные не отставали — и светофор показывал всё зелёным. Мониторинг измерял не то, что заявлял. Это ровно тот класс ошибки, который делает DQ-панель декоративной.

Part 3. Управление изменениями: пересчёт Gen5v1

3.1 Что это за класс изменения

Пересчёт модели — изменение **данных без изменения схемы**. Ничего не падает, ни один пайплайн не сбоят, ни один запрос не выдаёт ошибку. Все дашборды просто тихо начинают показывать другие числа. Именно поэтому такое изменение опаснее сноса таблицы: снос замечен через минуту, а сдвиг EL LT на 35% для винтажей первого полугодия 2025 — через квартал, на разборе отклонений.

3.2 Оценка влияния — запрос, а не обзвон

Без реестра потребителей «уведомить downstream» означает обзвонить всех, а на практике — не обзвонить никого. В реестре у каждого потребителя записано, *сколько дней уведомления* ему нужно и *что именно* для него ломается — потому что не всякое изменение ломающее для всех.

Потребитель	Уведомление	Что ломает именно его
Модель решений по CLIP	30 дней	Сдвиг EL LT — модель на нём калибрована
Месячная отчётность (CUBE)	20 дней	Изменение значений в закрытых периодах; новый снимок — нормально
Дашборд статементного COR	10 дней	Любое изменение значений прогноза или зерна
LLM-агент	0 дней	Переименование или удаление колонки. Изменение значений требует обновления контекста, а не уведомления

Строка про агента принципиальна: у него другой профиль хрупкости. Человека надо предупредить заранее, чтобы он успел перестроить отчёт; агенту уведомление бесполезно — ему нужно, чтобы контекст-слой обновился одновременно с данными.

3.3 Порядок действий и лестница Change Management

Степень	Что должно происходить	Реализовано
Basic	Изменение случается, о нём узнают постфактум	исходное состояние
Defined	Есть журнал изменений с признаком «ломающее» и автором	CHANGE_LOG , 4 записи, 3 ломающих
Controlled	Оценка влияния до релиза; версионирование; окно параллельной работы; уведомление по реестру	вью влияния; вью «текущая версия» и «предыдущая версия» для закрепления; надгробие вместо исчезнувшего объекта
Continuing	Незаложенное изменение обнаруживается автоматически	снимки DDL и сверка хеша с журналом

Верхняя ступень заслуживает пояснения. Скрипт пересборки, который поддерживают руками, расходится с реальностью — а расхождение документации и реальности является предметом всего этого задания. Поэтому DDL генерируется из живого аккаунта (95 КБ по десяти базам) и хранится со снимками. Изменившийся хеш при отсутствии записи в журнале даёт прямой вердикт: «схема сдвинулась, изменение не зарегистрировано». Это и есть измеримое определение слабого change management вместо декларативного.

3.4 Конкретно для Gen5v1

1. **Измерить** сдвиг до объявления: ×1.35 по EL LT для винтажей 2025-01...2025-06. Формулировка «модель пересчитана» без цифры бесполезна.
2. **Уведомить по реестру**: модель CLIP за 30 дней, отчётность за 20, дашборды за 10. Агенту — обновление контекста в тот же день.
3. **Версионировать**: старый снимок остаётся доступен через отдельную вью. Потребитель, которому нужна непрерывность ряда, закрепляется на прежней базе на оговорённый срок вместо спора о том, чьи цифры верные.
4. **Пометить срок вывода** старой вью из эксплуатации и отслеживать, кто ещё на ней.
5. **Записать** в журнал изменений с признаком «ломающее» и фактом уведомления — вью влияния помечает нарушение процесса, если ломающее изменение ушло без уведомления.

Part 4. Готовность к работе LLM-агентов

4.1 Чего приложениям не хватает для агента

Приложения А–Н — хорошая документация **для человека**. Агенту не хватает трёх вещей.

- **Проза вместо структуры.** «Фильтруйте по `account_type`» — предложение в тексте. Агенту нужна строка, которую можно найти запросом по имени объекта.
- **Нет разрешения терминов.** Глоссарий определяет, что `TDC` — это кредитная карта, но не сообщает, что в запросе это `product_risks = 'CC'`. Человек достроит, агент — нет.
- **Правила рядом с данными, а не внутри них.** Обязательный фильтр, записанный в документе, агент может проигнорировать. Обязательный фильтр, зашитый в метрику, проигнорировать нельзя.

4.2 Что построено

Объект	Записей	Что даёт агенту
Карточки таблиц	9	Назначение, зерно, колонка фильтра, семантика даты, знак, обязательные фильтры и раздел «чего не делать»
Граф джойнов	10	Точное выражение связи и ловушка каждого ребра. Иначе агент угадывает ключ
Грабли	10	Симптом, правило и исполняемый запрос-детектор
Глоссарий	17	Термин, синонимы и разрешение в объект/колонку/значение
Контракты метрик	4	Единственный разрешённый способ считать; легаси-формулы видны со статусом
Золотой набор вопросов	16	Регресс-тест с эталонами, посчитанными из данных

Весь контекст-слой — **13 664 символа, около 3 400 токенов**. Он помещается в системный промпт целиком, поэтому вопрос «а влезет ли документация» просто не возникает.

Проверка на живой модели. Один и тот же вопрос — «по какой колонке фильтровать кредитные карты в `cor_challenger` и что будет при ошибке» — задан модели внутри Snowflake Cortex дважды.

Без контекста: «фильтруйте по колонке *Card Type*» — колонки не существует.
С контекстом (два фрагмента, ~500 символов): «фильтруйте по `ACCOUNT_TYPE`; при `PRODUCT_RISKS` получите ноль строк без ошибки, потому что это остаточная колонка, всегда пустая».

Разница не в модели и не в промпте — в наличии контекст-слоя.

4.3 Что нужно, прежде чем агенту можно доверить рисковые данные

1. **Ограничение правами, а не инструкцией.** Роль агента не получает ни сырых слоёв, ни операционных данных, ни PII. До персональных данных агент не доходит не потому, что ему запретили, а потому что не может. Маскирование — вторая линия, не первая.
2. **Измеримая правильность.** Без закреплённого набора фраза «агент стал хуже после правки контекста» — неподтверждаемое ощущение. Доверие — это доля правильных ответов на фиксированном наборе, а не качество промпта. Два вопроса из шестнадцати проверяют не расчёт, а поведение: на «дай демографию должников» ожидается **отказ** со ссылкой на отсутствие доступа, а не обход через другие таблицы; на обращение к дропнутой таблице — **перенаправление**, а не падение.
3. **Аудит постфактум.** Все запросы роли агента видны в истории с текстом, объёмом и стоимостью. Доверие к автономному агенту — не свойство промпта, а возможность посмотреть, что он на самом деле сделал.
4. **Только чтение** и канон, встроенный в семантическую модель.

4.4 Что нужно, чтобы поднять каталог до уровня Controlled

Признак	Сейчас	Что доделать
Описание каждого объекта	54 из 54	сделано; держать через проверку «описание короче 20 символов»
Владелец и стюард	14 из 54	довести tier1 и tier2 до 100%, tier3 — сознательно нет
Реестр потребителей	10 связей	покрыть все дашборды Tableau и регулярные выгрузки
Линидж	частично	достроить рёбра для объектов, которые заполняются процедурами
Машиночитаемость	да	каталог — таблицы, а не вики-страница: их одинаково читают человек, дашборд и агент

Техническое ограничение, которое стоит знать при выборе инструментов. Системная вью Snowflake со ссылками на теги отстаёт до двух часов — сразу после разметки она возвращает ноль строк (проверено). Каталог, отстающий на два часа, каталогом не является. Поэтому теги остаются нативным механизмом для линиджа и политик, а живой каталог читает системную схему и собственные реестры.

Part 5. Оценка зрелости и план на 90 дней

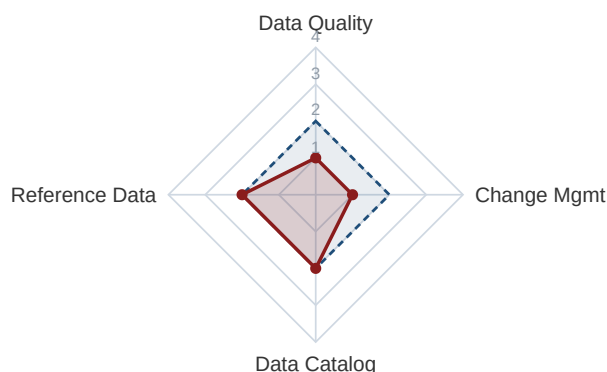
5.1 Где сейчас находится Plata

Appendix I разрешает взять определения DMBoK либо свою рабочую трактовку, но требует указать, какую. **Использую собственную рабочую трактовку** — она операционнее для этих четырёх дисциплин и проверяема наблюдением, а не самооценкой:

Уровень	Рабочее определение, единое для всех четырёх дисциплин
Basic	Практика существует в головах. Ни порогов, ни владельца, ни следа. Проблема обнаруживается пользователем
Defined	Правило записано и у него есть владелец. Соблюдение проверяется вручную и выборочно
Controlled	Соблюдение проверяется автоматически, нарушение адресно эскалируется, отклонение фиксируется до того, как о нём узнает потребитель
Continuing improvement	Измеряется сама система контроля: доля инцидентов, пойманных раньше пользователя, доля ложных срабатываний, время до исправления

Оценка ниже — по свидетельствам из приложений А–Н, а не по общему впечатлению.

Дисциплина	Уровень	Свидетельство	Насколько уверен и что бы собрал
Data Quality Management	Basic	Приложения перечисляют ожидаемую свежесть таблиц, но нигде нет ни порогов, ни владельца сигнала, ни следа автоматической проверки. Формулировка «expecting weekend dates will always appear as stale» показывает, что ложные тревоги известны и с ними живут.	Уверен. Свидетельство прямое. Дособрал бы одно: журнал инцидентов за квартал — чтобы отличить «проверок нет» от «проверки есть вне документации»
Change Management	Basic	Дропнутая таблица описана как факт с инструкцией «обновите запросы». Пересчёт модели — как предупреждение «имейте в виду». Ни уведомления, ни версионирования, ни оценки влияния.	Уверен. Проверил бы переписку по дропу PV_DETAILS : если уведомление было, но вне процесса, это Defined с нерабочей фиксацией, а не Basic
Data Catalog	Defined	Документация существует, структурирована и содержательна — это заметно выше нуля. Но она для человека, не версионирована рядом с данными и не покрывает владение.	Свидетельств мало. Оценка опирается на сами приложения. Нужно: ведётся ли документация в git, кто её обновляет и по какому триггеру. Если обновление ручное и нерегулярное — оценка сползает к Basic. Пять оборванных перекрёстных ссылок (см. приложение-ревизию) — довод в пользу Basic
Reference Data Management	Defined	Коды продуктов согласованы внутри слоёв, расхождения (TDC , GRZD_CLIPPED в составе BP «СС») известны и задокументированы. Единого справочника нет.	Свидетельств мало и они косвенные. Видны только продуктовые коды. Не видны справочники причин списания, кодов коллекшна, каналов. Собрал бы инвентарь всех enum-полей в dm.* и проверил, у скольких есть источник истины — это и есть настоящий замер RDM



■ **сейчас** — Data Quality 1, Change Management 1, Data Catalog 2, Reference Data 2

▤ **цель 90 дней** — Data Quality 2, Change Management 2; остальные без изменений *сознательно*

Шкала: 1 Basic · 2 Defined · 3 Controlled · 4 Continuing improvement. Форма фигуры и есть ответ на вопрос приоритизации: провал по вертикальной оси — те две дисциплины, где отсутствие контроля обходится дороже всего, и именно их я двигаю первыми.

5.2 Две дисциплины, которые я двигаю первыми

Data Quality Management и Change Management. Обоснование не только в том, что задание называет их самой острой болью, но и в порядке зависимостей: без DQ невозможно доказать, что изменение что-то сломало, а без управления изменениями любое улучшение качества откатывается следующим релизом.

Дисциплина	Что значит «на ступень выше» конкретно	Критерий приёмки на 90-й день
DQM Basic → Defined	Все таблицы tier1 покрыты проверками свежести, полноты и уникальности; свежесть статементных таблиц измеряется в рабочих днях; у каждой проверки есть порог, стюард и предписанное действие	Ноль ложных тревог по понедельникам; каждый инцидент имеет владельца; доля инцидентов, найденных мониторингом раньше пользователя, измеряется и растёт
Change Management Basic → Defined	Журнал изменений с признаком «ломающее»; реестр потребителей с требуемым сроком уведомления; версионирование витрин при пересчёте моделей	Ни одного ломающего изменения без записи и уведомления; сверка снимков схемы не находит незалогированных изменений

Параллельно — **канонический метрик-слой**. Формально это Metadata, но без него не работает ни одна из двух: нельзя мониторить качество метрики, определение которой не зафиксировано. Мост между расчётами — минимальная версия, которая окупается в первый месяц.

5.3 Что я меряю в первые 30 дней

1. **Сколько версий одной метрики реально в обращении.** Ожидаю больше трёх: три из задания плюс варианты в Excel. Если их две — приоритет метрик-слоя падает.
2. **Доля инцидентов качества, найденных мониторингом раньше пользователя.** Стартовая точка, скорее всего, близка к нулю. Это главная метрика прогресса по DQM.
3. **Число изменений схемы за квартал и доля зарегистрированных.** Считается сверкой снимков DDL. Если незарегистрированных мало — Change Management спускается ниже в приоритете, и я перекладываю усилие в каталог.
4. **Время на ответ на типовой вопрос аналитика.** Прокси-метрика полезности контекст-слоя, замеряется до и после.
5. **Доля правильных ответов агента на золотом наборе.** Стартовый замер до внедрения контекст-слоя.

Пункты 1 и 3 — это ревизия приоритетов, а не подтверждение: они могут перевернуть план, и это нормально.

5.4 Что сознательно не является приоритетом на 90 дней

- **Reference Data Management.** Расхождения кодов известны и локализованы; минимальный справочник с алиасами закрывает 90% боли за день работы. Полноценный RDM на 90 днях вытеснит DQM без соразмерной отдачи.
- **Master Data Management по клиентам.** Требуется согласования за пределами риска и не разблокирует ни одну из двух приоритетных дисциплин.
- **Смена платформы или закупка каталога.** Нативных механизмов Snowflake достаточно для перехода Basic → Defined. Инструмент, купленный до того, как процесс работает, консервирует отсутствие процесса.
- **Полное покрытие владением всех объектов.** tier1 и tier2 — да, tier3 — осознанно нет: разметка сорока технических таблиц создаёт видимость governance без его содержания.

Уточнения к приложениям

Четыре наблюдения, полученные при воспроизведении описанного ландшафта. Приложения ими не опровергаются — уточняются.

Утверждение приложения	Что показала проверка	Как я это трактую
Неверная колонка фильтра «молча возвращает пусто» (A, D)	В Snowflake обращение к несуществующей колонке даёт ошибку компиляции . Молчаливым промах бывает, только если колонка существует	Симптом описан верно, значит в реальной таблице колонка осталась после переименования. В песочнице я воспроизвёл именно это; стоит подтвердить на боевой системе и явно пометить остаточную колонку как устаревшую
Фильтр <code>statement_num >= 2</code> — ключевая грабля (C, D)	Вклад 0.05–0.24 п.п. против 2.1–2.7 п.п. у выбора зерна	Предупреждение верное, приоритет расставлен неверно. В контекст-слое обе грабли присутствуют, но зерно вынесено выше
Помесячные результаты Gen2 «очень близки» к Gen3 (G)	Подтверждается: вклад поколения модели 0.4–1.7 п.п., меньше вклада зерна	Важное уточнение к постановке Part 2: спор трёх команд порождён методическими решениями , а не расхождением моделей
BP моделирует провизии по 13 бакетам DPD 30...390 (C)	В воспроизведённых данных заполнено 8 из 13	Следствие квартальной механики списаний: до поздних бакетов доживают единицы счетов. Схему поддерживать на все 13 нужно, ожидать данных во всех — нет

Допущения

- Задание говорит о шести базах, приложение A перечисляет девять префиксов. Строил по приложению A.
- Колоночный состав ODS-таблиц, заявок, скоринга и dbt-схем в приложениях не описан — сконструирован минимально достаточным и помечен в реестре объектов.
- Абсолютные величины (число счетов, уровень COR, ставки резервов) правдоподобны, но синтетические. Пропорции и соотношения между расчётами — то, что имеет значение.
- «Апрель» из Part 2 трактован как апрель 2026.
- Оценка зрелости опирается только на приложения A–I; интервью с командами может её сдвинуть.

Приложение: воспроизводимость

Песочница собирается с нуля одиннадцатью скриптами примерно за десять минут и стоит 2.67 кредита. Любое утверждение этого документа проверяется одним запросом: корректность сборки — `RISK_GOV.DQ.BUILD_ASSERTIONS`, мост между расчётами — `V_COR_RECONCILIATION`, светофор качества — `V_SCORECARD`, каталог — `V_OBJECT_CATALOG`, оценка влияния изменения — `V_CHANGE_IMPACT`, незарегистрированные изменения схемы — `V_SCHEMA_DRIFT`, контекст для агента — `AGENT.V_AGENT_CONTEXT`.

Объём песочницы: 10 баз, 23 схемы, 59 таблиц, 15 вью, 1 семантическая вью, 8 141 114 строк, 0.307 ГБ. Скрипты и документация — в репозитории, каталог `snowflake-sandbox/`.